DATA STRUCTURE AND ALGORITHMS

LESSON 05
LINKED LIST - PART 2

Conceptual Difficulty

Implementation Difficulty

LECTURER: SEAK LENG

OVERVIEW

1. Introduction to algorithm
2. Basic data types and statements
3. Control structures and Loop
4. Array
5. Data structure
6. Sub-programs
7. Recursive
8. Pointers
9. *Linked Lists*
10. Stacks and Queues
11. Sorting algorithms
12. Trees

C++

OUTLINE

- How to implement linked list in C++
  - Delete an element from the list
  - Reverse
  - Combine
  - Sort
- Examples

LIST OPERATIONS

- ✓Creating a list
- ✓Insert a new element to a list
- ✓Search an element in a list
- ✓Update an element in a list
- ✓Delete an element from a list
- ✓Reverse a list
- ✓Sort a list using pointer manipulation
- ✓Combine two lists

Singly Linked List (SLL)

SINGLY LINKED LIST

- Create a linked list

1. Define a node structure: data, next pointer
2. Define a list structure: head, tail, n
3. Create an empty list

Steps to create an empty list:

1. Create a list variable
2. Allocate memory
3. Set 0 to n since we are creating an empty list
4. Head points to **NULL or nullptr**
5. Tail points to **NULL or nullptr**

```cpp
#include<iostream>
using namespace std;
struct Element{
    int data;
    Element *next;
};
```

```cpp
struct List{
    int n;  //number of elements
    Element *head;
    Element *tail;
};
```

```cpp
//A function to create an empty list
List* createList(){
    List *ls;

    ls = new List(); //allocate memory
    //ls.n = 0; //error
    ls->n = 0;
    ls->head = NULL;
    ls->tail = NULL;

    return ls;
}
```

C++ Procedural Code for Singly Linked List

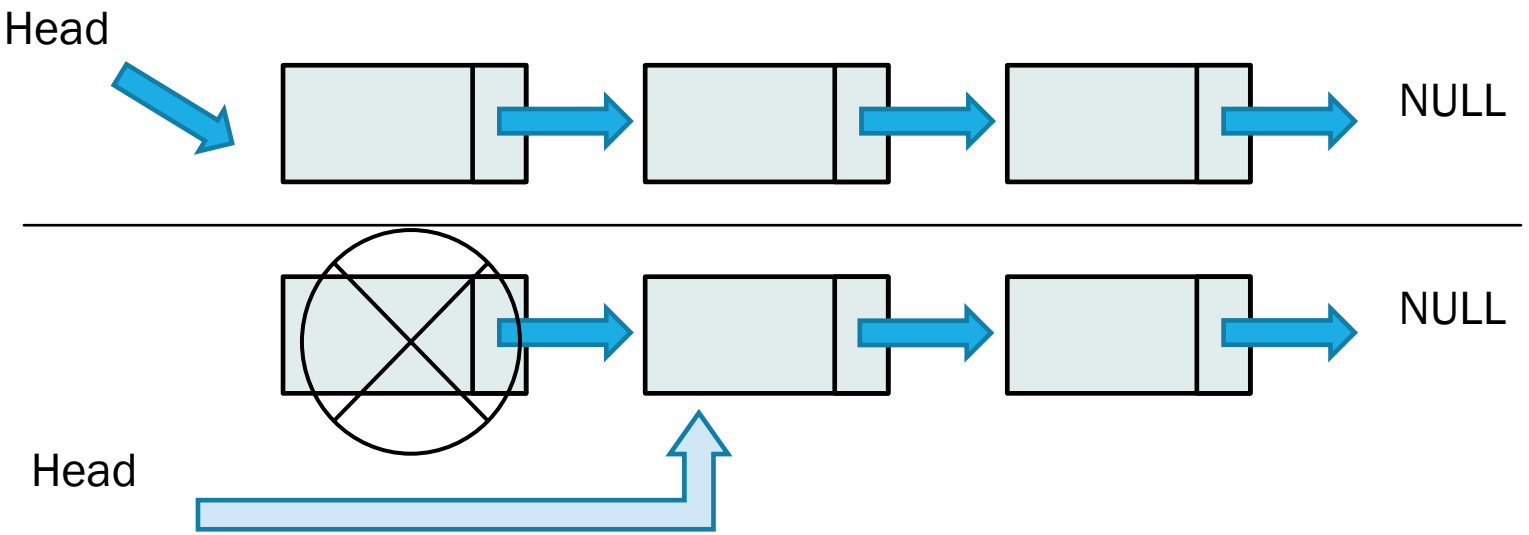
SINGLY LINKED LIST

- Delete an element from a list
  - Delete at the beginning
  - Delete at the end
  - Delete at the position

How to delete data from linked list

DELETION AT THE BEGINNING OF THE LIST

- Make the pointer of the head points to the 2nd element
- Delete the original first element using *delete* keyword
- Update the tail if the list becomes empty



DELETION AT THE BEGINNING OF THE LIST

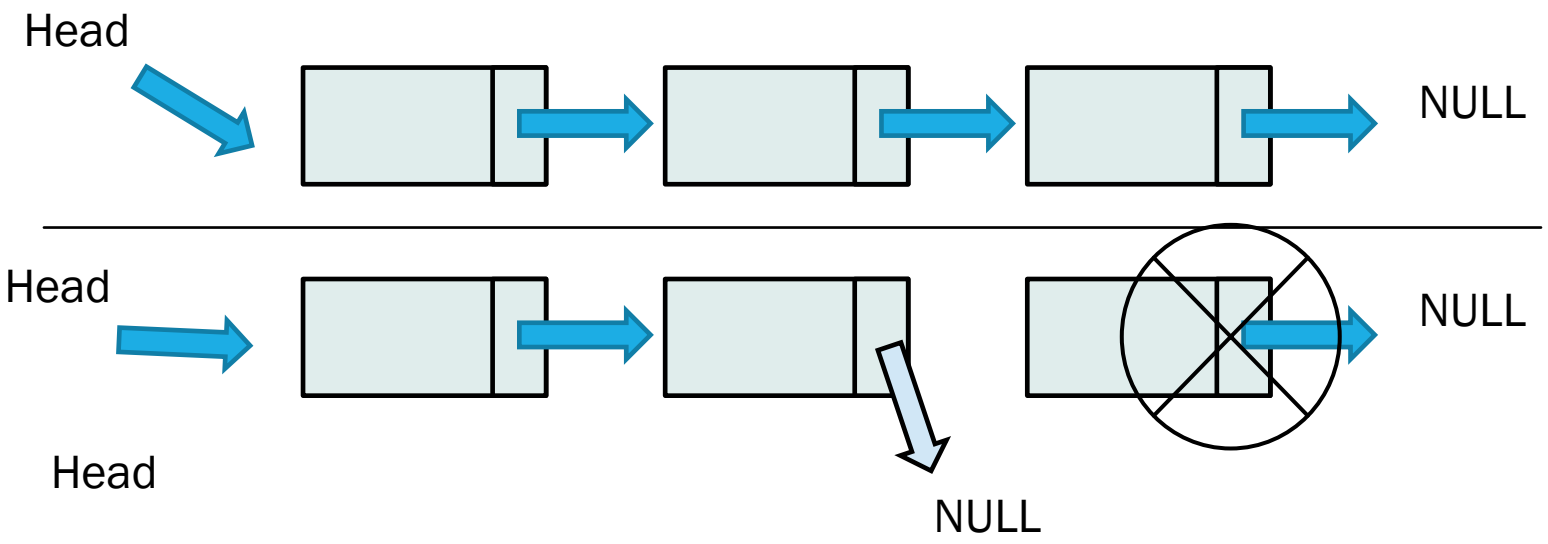
- Case 1: Empty list. Exit;
- Case 2: Non-empty list
  - Store current head in a temp pointer
  - Move head to the next node
  - Delete old head node
  - If list became empty, update tail to null pointer
  - Decrease the number of nodes by 1

```
void deleteBeg(List* ls){
    if(ls->n==0) return;

    Element* tmp = ls->head;
    ls->head = ls->head->next;
    delete tmp;
    if(ls->n==1){
        ls->tail = nullptr;
    }
    ls->n--;
}
```

DELETION AT THE END OF THE LIST

- Make the pointer of the second last element points to NULL
- Delete the last node using *delete* keyword



DELETION AT THE END OF THE LIST

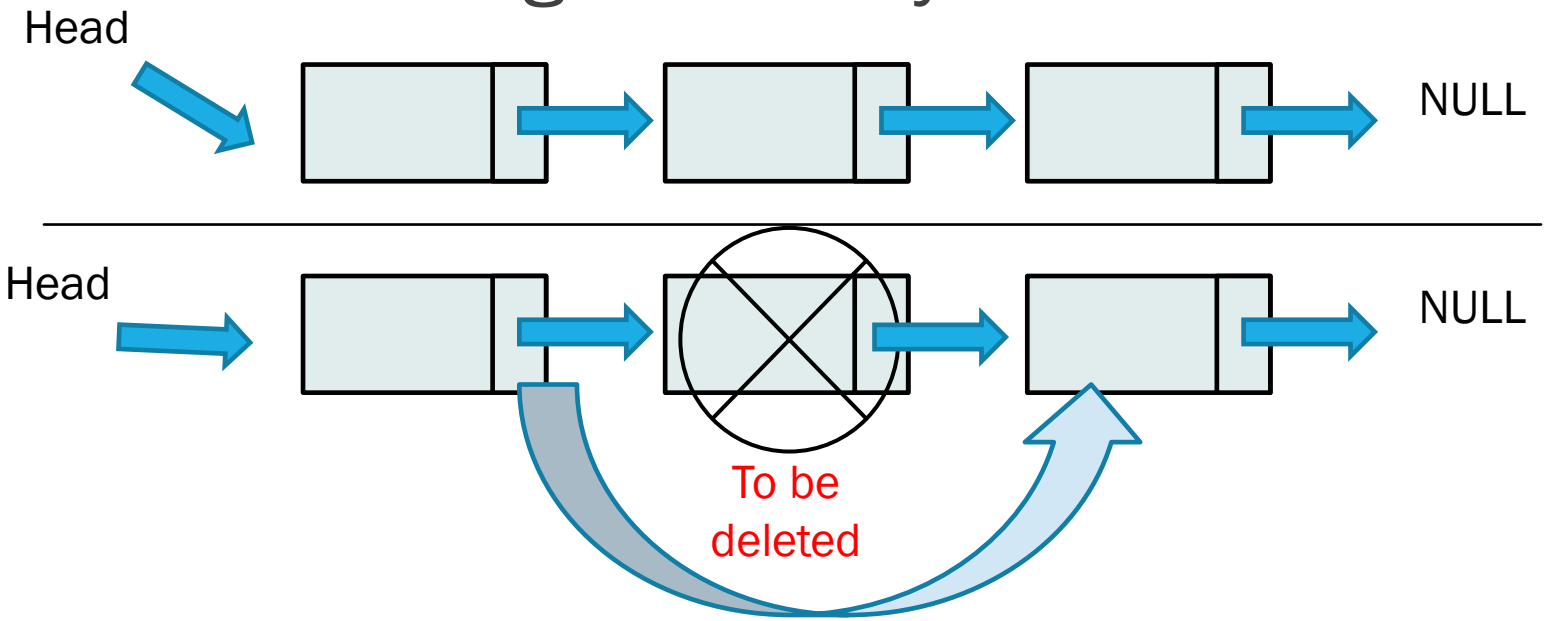
- Case1: Empty list. Exit
- Case 2: the list has only one node. delete the tail, set head and tail to null
- Case 3: Multiple nodes.
  - initialize a temporary pointer e pointing to head,
  - traverse the list until the second last node
  - Set its next to null pointer (e->next = nullptr)
  - Delete the tail node
  - Update the tail to point to the new last node e
- Decrease the number of nodes by 1

```
void deleteEnd(List* ls){
    if(ls->n==0) return;
    if(ls->n==1){
        delete ls->tail;
        ls->head = ls->tail = nullptr;
    }else{

        Element* e = ls->head;
        while(e->next != ls->tail){
            e = e->next;
        }
        e->next = nullptr;
        delete ls->tail;
        ls->tail = e;
    }
    ls->n--;
}
```

DELETION AT POSITION

- Make the pointer of the predecessor element of E points to the successor element of E
- Delete the node using *delete* keyword



DELETION AT POSITION

- Case 1: Empty list or invalid position. Return
- Case 2: Position is 0, use deletion at the beginning
  - Store current head in a temp pointer
  - Move head to the next node
  - Delete temp pointer
  - If list became empty, update tail to null pointer
  - Decrease the number of nodes by 1
- Case 3: Other positions
  - Traverse to the node at given position
    - Loop from i=0 to **pos – 1**
  - Delete the node at position
    - Temp = current->next;
    - Current->next = temp->next;
    - If temp is the last element, update the tail to current
    - Delete temp
  - Decrease the number of nodes by 1

DELETION AT POSITION

```
void deleteAtPos(List* ls, int pos){
    if(ls->n==0 || pos<0 || pos>=ls->n){
        cout << "Cannot delete" << endl;
        return;
    }
    if(pos==0){
        Element* tmp = ls->head;
        ls->head = tmp->next;
        delete tmp;
        if(ls->n==1){
            ls->tail = nullptr;
        }
    }else{
        Element* current = ls->head;
        for(int i=0;i<pos-1;i++){
            current = current->next;
        }
        Element* tmp = current->next;
        current->next = tmp->next;
        if(tmp == ls->tail){
            ls->tail = current;
        }
        delete tmp;
    }
    ls->n--;
}
```

```cpp
#include <iostream>

using namespace std;

struct Element{
    int data;
    Element* next;
};

struct List{
    int n;
    Element* head;
    Element* tail;
};

List* createEmptyList(){
    List* ls = new List;
    ls->head = nullptr;
    ls->tail = nullptr;
    ls->n = 0;
    return ls;
}
```

```cpp
void addEnd(List* ls, int data){
    Element* e = new Element;
    e->data = data;
    e->next = nullptr;
    if(ls->n==0){
        ls->head = e;
    }else{
        ls->tail->next = e;
    }
    ls->tail = e;
    ls->n++;
}
void deleteEnd(List* ls){
    if(ls->n==0) return;
    if(ls->n==1){
        delete ls->tail;
        ls->head = ls->tail = nullptr;
    }else{
        Element* e = ls->head;
        while(e->next != ls->tail){
            e=e->next;
        }
        e->next = nullptr;
        delete ls->tail;
        ls->tail = e;
    }
    ls->n--;

}
```

```cpp
void display(List* ls){
    Element* e = ls->head;
    while(e!=nullptr){
        cout << e->data << " ";
        e = e->next;
    }
    cout << endl;
}

void deleteBeg(List* ls){
    if(ls->n==0) return;

    Element* tmp = ls->head;
    ls->head = ls->head->next;
    delete tmp;
    if(ls->n==1){
        ls->tail = nullptr;
    }
    ls->n--;
}
```

```
void deleteAtPos(List* ls, int pos){
    if(ls->n==0 || pos<0 || pos>=ls->n){
        cout << "Cannot delete" << endl;
        return;
    }
    if(pos==0){
        Element* tmp = ls->head;
        ls->head = tmp->next;
        delete tmp;
        if(ls->n==1){
            ls->tail = nullptr;
        }
    }else{
        Element* current = ls->head;
        for(int i=0;i<pos-1;i++){
            current = current->next;
        }
        Element* tmp = current->next;
        current->next = tmp->next;
        if(tmp == ls->tail){
            ls->tail = current;
        }
        delete tmp;
    }
    ls->n--;
}
```

```
int main(){
    List* list1 = createEmptyList();
    for(int i=0;i<10;i++){
        addBeg(list1, i);
    }

    deleteBeg(list1);
    display(list1);
    deleteEnd(list1);
    display(list1);

    for(int i=0;i<10;i++){
        deleteBeg(list1);
        display(list1);
    }

    deleteAtPos(list1, 9);
    display(list1);
}
```

REVERSE THE LIST

- Initialize 3 pointers
  - prev = nullptr;
  - current = head;
  - next = nullptr;
- Update tail = head.
- Traverse the list, for each node
  - Store the next node to the current->next
  - Reverse the link: current->next = prev
  - Move prev and current:
    - Prev = current
    - Current = next
- After the loop, prev will be the new head of the list. Update head = prev.

```
void reverseList(List* ls){
    Element* prev = nullptr;
    Element* current = ls->head;
    Element* next = nullptr;

    ls->tail = ls->head;

    while(current!=nullptr){
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }

    ls->head = prev;
}
```

COMBINE TWO LISTS

- If list2 is empty: return
- If list1 is empty:
  - Set list1->head = list2->head
  - Set list1->tail = list2->tail
  - Set list1->n = list2->n
- Otherwise:
  - Link the tail of list1 to the head of list2
  - Set the tail of list1 to point to the tail of list2
  - Sum the number of elements in the list1 and list2

```
void combine(List* list1, List* list2){
    if(list2->n==0){
        return;
    }
    if(list1->n==0){
        list1->head = list2->head;
        list1->tail = list2->tail;
        list1->n = list2->n;
    }
    else{
        list1->tail->next = list2->head;
        list1->tail = list2->tail;
        list1->n += list2->n;
    }
}
```

SORT A SINGLY LINKED LIST USING BUBBLE SORT

Bubble sort is a simple sorting algorithm that repeatedly:

1. Compares adjacent elements
2. Swaps them if they're in the wrong order
3. Continues this process until the entire list is sorted

```
void sortList(List* ls){
    if(ls->n<2) return;
    Element* ptr ;
    Element* lastptr = nullptr;
    bool swapped;
    do{
        ptr = ls->head;
        swapped = false;
        while(ptr->next != lastptr){
            if(ptr->data > ptr->next->data){
                int temp = ptr->data;
                ptr->data = ptr->next->data;
                ptr->next->data = temp;
                swapped = true;
            }
            ptr = ptr->next;
        }
        lastptr = ptr;
    }while(swapped);
}
```

PRACTICE

1. Create a singly linked list that can store integer numbers. Then
   a. Create two data structure (Element and List), and create an empty singly linked list
   b. Create function to add the element to the end of the list
   c. Create function to add the element to the beginning of the list
   d. Create function to display all numbers in the list
   e. Create function to delete the element from the beginning of the list
   f. Create function to delete the element from the end of the list

PRACTICE

2. Create a singly linked list that can store id, names of students.
   a. Create structure of student, element, list, and create an empty singly linked list.
   b. Create a function to add the student's info to the beginning of the list
   c. Display all students in the list
   d. Create a function to reverse students in the list
   e. Create a function to sort students by their score in the list
   f. Create a function to combine two student lists

Q&A